

计算机组成原理

组成
原理

组成原理26春

哈尔滨工业大学（深圳）



扫描群二维码，立刻加入该群

该二维码 1 年内 (2027/3/4前)有效

王鸿鹏
计算机科学与技术学院
wanghp@hit.edu.cn

第二章 计算机中数的表示和运算方法

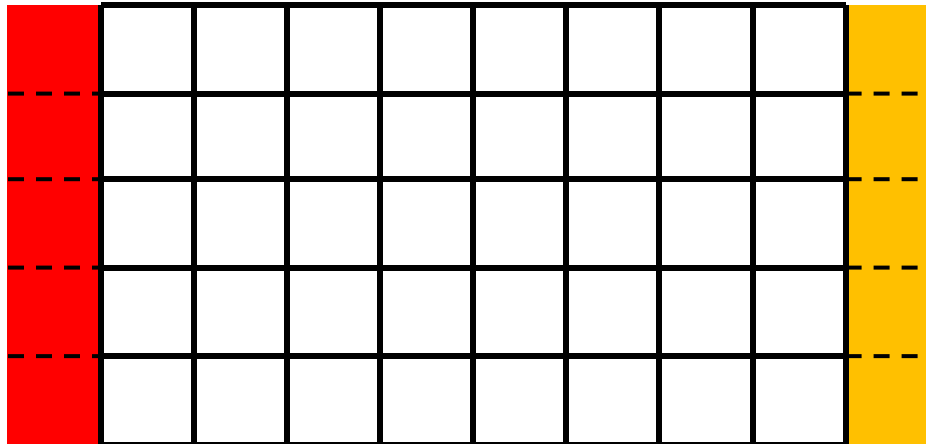
- 计算机中数的表示
 - 无符号数和有符号数
 - 定点表示和浮点表示
 - IEEE754标准
- 计算机中的运算方法
 - 算术移位与逻辑移位
 - 算术运算

计算机中数的表示

- **无符号数和有符号数**
- 定点表示和浮点表示
- IEEE754标准

无符号数

- 数字电子计算机中的数是**离散的**。
- 寄存器的位数反映无符号数的表示范围。



8 位



0 ~ 255

16 位



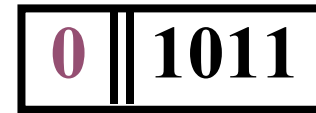
0 ~ 65535

有符号数：真值与机器数（机器码）

真值：带符号的数

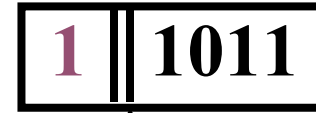
机器数：符号数字化的数

+ 0.1011或0.1001



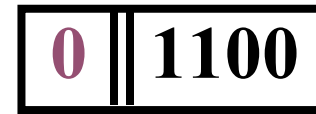
小数点的位置

- 0.1011



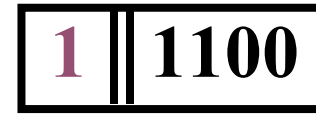
小数点的位置

+ 1100或1100



小数点的位置

- 1100



小数点的位置

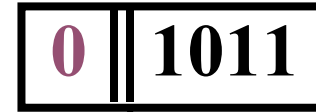
注：以后非特殊说明，**默认二进制**数表示；
位数不是8的倍数，只是为了讲解方便；
机器数一般比真值多一位符号位。

有符号数：真值与机器数

真值：带符号的数

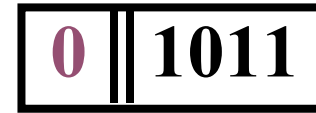
机器数：符号数字化的数

+ 0.1011



小数点的位置

+ 1011



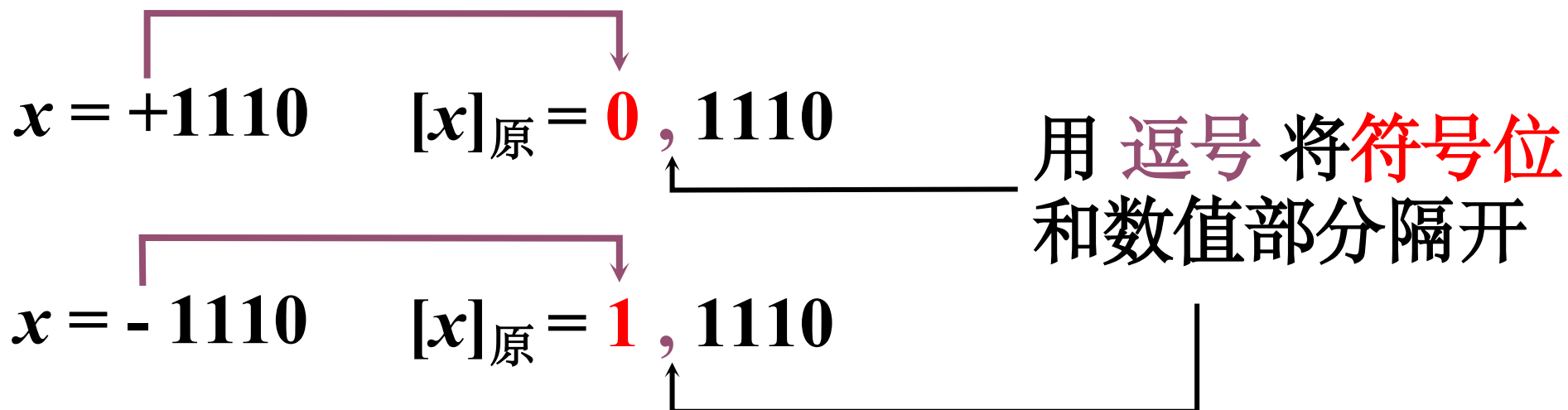
小数点的位置



- 小数和整数在计算机中的表示一模一样?
- 如何表示既有整数又有小数的数据，如+11.01?

原码表示法：整数

带符号的绝对值表示



$$[x]_{\text{原}} = 2^4 + 1110 = 2^4 - (-1110)$$

$$[x]_{\text{原}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^n - x & 0 \geq x > -2^n \end{cases}$$

x 是真值, n 是数值位数

原码表示法：小数

假设：真值的数据位数就是寄存器的数据位数

$$x = +0.1101 \quad [x]_{\text{原}} = 0.1101$$

用 **小数点** 将符号位和数值部分隔开

$$x = -0.1101 \quad [x]_{\text{原}} = 1.1101 = 1 - (-0.1101)$$

$$x = +0.1000000 \quad [x]_{\text{原}} = 0.1000000$$

$$x = -0.1000000 \quad [x]_{\text{原}} = 1.1000000 = 1 - (-0.1000000)$$

$$[x]_{\text{原}} = \begin{cases} x & 1 > x \geq 0 \\ 1 - x & 0 \geq x > -1 \end{cases} \quad x \text{ 为真值}$$



例：根据原码求真值

1. 已知 $[x]_{\text{原}} = 0.1101$, 求 x

解：根据定义 $\because [x]_{\text{原}} = 0.1101 > 0$

$$\therefore x = +0.1101$$

2. 已知 $[x]_{\text{原}} = 1.0011$, 求 x

解：由小数原码定义得

$$x = 1 - [x]_{\text{原}}$$

$$= 1 - 1.0011 = -0.0011$$

0的原码?

注意: 要分成正负、小数和整数分别讨论

• 例: 求小数 $x = 0.0$ 的原码(假设数值位是4位)

解: 设 $x = +0.0000$ 则 $[+0.0000]_{\text{原}} = 0.0000$

$x = -0.0000$ 则 $[-0.0000]_{\text{原}} = 1.0000$

同理, 对于整数(假设数值位是4位)

$[+0]_{\text{原}} = 0,0000$

$[-0]_{\text{原}} = 1,0000$

∴ 正零的原码和负零的原码不同

原码的优缺点

- 优点：简单、直观
- 缺点：1) +0和-0原码不一样
2) 作加法运算时，会出现如下问题：

要求	数1	数2	实际操作	结果符号
加法	正	正	加法	正
加法	正	负	减法	正或负
加法	负	正	减法	正或负
加法	负	负	加法	负

实际运算时能否只作加法运算？

原码的缺点——续

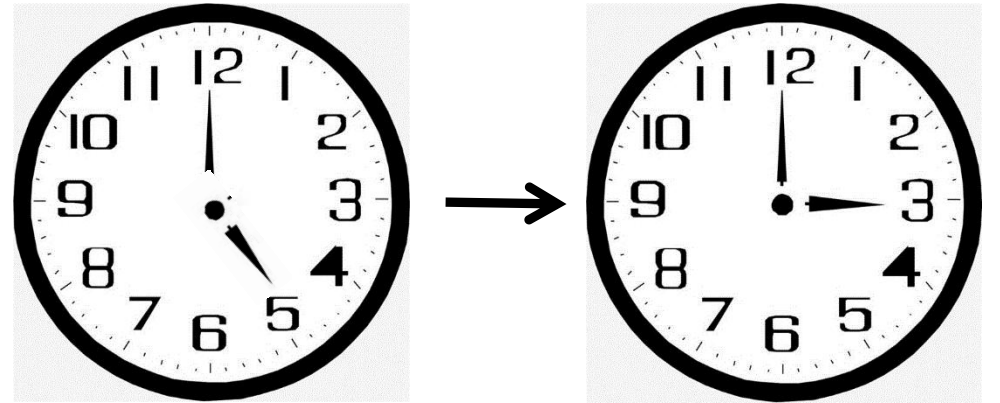
十进制		原码									
	3		<table border="1"><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	0	0	0	0	0	0	1	1
0	0	0	0	0	0	1	1				
+	-3	+	<table border="1"><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	1	0	0	0	0	0	1	1
1	0	0	0	0	0	1	1				
	0		<table border="1"><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	1	0	0	0	0	1	1	0
1	0	0	0	0	1	1	0				
		-6	← 转十进制								



原码运算 不等于 十进制运算?

补数表示法

- 逆时针: $5 - 2 = 3$
- 顺时针: $5 + 10 = 3 + 12$
- 时钟以12为模



- 可见 -2 可用 $+10$ 代替
 - 称 $+10$ 是 -2 (以 12 为模) 的补数
 - 记作 $-2 \equiv +10 \pmod{12}$
- 同理 $-4 \equiv +8 \pmod{12}$
- $-5 \equiv +7 \pmod{12}$

减法 $\longrightarrow$ 加法

补数——续

减法计数器1011 $\longrightarrow$ 0000 ?

$$\begin{array}{r} 1011 \\ - 1011 \\ \hline 0000 \end{array}$$

$$\begin{array}{r} 1011 \\ + 0101 \\ \hline 10000 \end{array}$$

(加法计数器)

可见-1011 与 + 0101 作用等价 (模16)

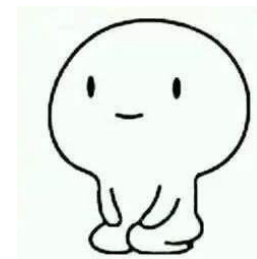
自然舍去

记作 $-1011 \equiv + 0101 \pmod{2^4}$

同理 $-011 \equiv + 101 \pmod{2^3}$

$-0.1001 \equiv + 1.0111 \pmod{2^1}$

正数?补数?



- 结论(真值的绝对值小于模)
 - 一个负数加上 “**模**” 即得该负数的补数
 - 一个负数和一个正数互为补数时, **绝对值之和**即为模数

补数——续

正数的补数即为其本身

对于时钟：3点、15点、27点都是3点 $\longrightarrow$ $3 \equiv 15 \equiv 27 \pmod{12}$
 $3 \equiv 3+12 \equiv 3+24 \equiv 3 \pmod{12}$

同理： $+0101 \equiv +0101 + 2^4 \equiv +0101 \pmod{2^4}$

前面已证明： $-1011 \equiv -1011 + 2^4 \equiv +0101 \pmod{2^4}$

补数前面
增加符号位

$\boxed{+}0101 \longrightarrow 0, 0101$

$\boxed{-}1011 \longrightarrow 1, 0101$

$1011 + 10101$
 $+ 1011$

$\hline 1,00000 = 2^4 + 1$

$(\pmod{2^{4+1}})$

+0101究竟是一1011的补数
还是+0101的补数呢?



补码表示法： 二进制整数

$$[x]_{\text{补}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^{n+1} + x & 0 > x \geq -2^n \pmod{2^{n+1}} \end{cases}$$

其中： x 为真值， n 为二进制整数的位数

$$x = -1011000$$

$$\begin{aligned} x \text{ 的补数} &= -1011000 + 2^7 \quad \text{检验上式为什么是 } 2^{n+1}? \\ &= 0101000 \end{aligned}$$

$$x = +0101000$$

$$[x]_{\text{补}} = 0,0101000$$



用 逗号 将符号位
和数值部分隔开

$$[x]_{\text{补}} = 1,0101000$$

2^7

$$[x]_{\text{补}} = 2^{7+1} + (-1011000)$$

$$= 100000000$$

$$- \quad 1011000$$

$$\hline 1,0101000$$

补码表示法： 二进制（纯）小数

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \pmod{2} \end{cases}$$

其中： x 为真值

$$x = + 0.1110$$
$$[x]_{\text{补}} = 0.1110$$

用 小数点 将符号位
和数值部分隔开

$$x = - 0.1100000$$
$$\begin{aligned} [x]_{\text{补}} &= - 0.1100000 + 2 \\ &= 10.0000000 \\ &\quad - 0.1100000 \\ &= 1.0100000 \end{aligned}$$

求补码的快捷方式(适用绝大多数情况)

- 设 $x = -1010$, 求 $[x]_{\text{补}}$ 。

$$\begin{aligned} [x]_{\text{补}} &= 2^{4+1} - 1010 &&= 11111 + 1 - 1010 \\ &= 100000 &&= 11111 + 1 \\ &\quad - 1010 &&\quad - 1010 \\ \hline &= 1,0110 &&\quad \boxed{10101} + 1 \\ & &&= 1,0110 \end{aligned}$$

$$\text{又 } [x]_{\text{原}} = \boxed{1,1010}$$

- 当真值为 **负** 时, 根据原码求补码的快捷方法:

原码除符号位外, 每位取反, 末位加 1

自右向左从最先遇到的1左边开始, 每位取反(符号位除外)

原码、补码表示法小结

x 是真值, n 是小数点左边的数值位数 (纯小数 n 为0)

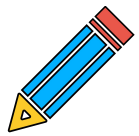
$$[x]_{\text{原}} = \begin{cases} 0x & 2^n > x \geq 0 \\ 2^n - x & 0 \geq x > -2^n \end{cases}$$

$$[x]_{\text{补}} = \begin{cases} 0x & 2^n > x \geq 0 \\ 2^{n+1} + x & 0 > x \geq -2^n \pmod{2^{n+1}} \end{cases}$$

符号位和数值位之间的 . 和 , 是给人看的, 机器中没有。
当真值为 负 时, 根据原码求补码的快捷方法:

原码除符号位外, 每位取反, 末位加 1

自右向左从最先遇到的1左边开始, 每位取反(符号位除外)



练习：求下列真值的补码

真值	$x = -70$ $= -1000110$	$x = -0.1110$	$x = 0.0000$	$x = -0.0000$	$x = -1.0000$
$[x]_{\text{原}}$	1,1000110	1.1110	0.0000	1.0000	不能表示
$[x]_{\text{补}}$	1,0111010	1.0010	0.0000	0.0000	1.0000

$$[+0]_{\text{补}} = [-0]_{\text{补}}$$

小数补码定义

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \pmod{2} \end{cases}$$

$$[-1.0000]_{\text{补}} = 2 + x = 10.0000 - 1.0000 = 1.0000$$

补码、原码间的互相转换

- 当真值为 **负** 时，根据**原码求补码**的快捷方法
 - 原码除符号位外，每位取反，末位加 1
 - 自右向左从最先遇到的1左边开始，每位取反(符号位除外)
- 逆运算：当真值为**负**时，根据**补码求原码**的快捷方法
 - **除符号位，末位减 1，每位取反**（等价于**每位取反，末位加1**）
 - 小数： $0.1...1 - (\text{原码} - 2^{-n}) = (0.1...1 - \text{原码}) + 2^{-n}$
 - 整数： $1...1 - (\text{原码} - 1) = (1...1 - \text{原码}) + 1$
- 补码→原码、原码→补码的**方法相同**!

已知补码求真值

已知 $[x]_{\text{补}} = 1,1110$, 求 x

解: 由定义得 $x = [x]_{\text{补}} - 2^{4+1}$

$$= 1,1110 - 100000$$

$$= -0010$$

或: $[x]_{\text{补}} \rightarrow [x]_{\text{原}} \quad [x]_{\text{原}} = 1,0010$

$$\therefore x = -0010$$

举例:已知小数补码求真值

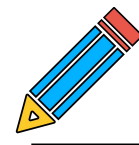
已知 $[x]_{\text{补}} = 1.0001$, 求 x 。

$$\begin{aligned}\text{解: 由定义得 } x &= [x]_{\text{补}} - 2 \\ &= 1.0001 - 10.0000 \\ &= -0.1111\end{aligned}$$

$$\begin{aligned}\text{或: } [x]_{\text{补}} &\rightarrow [x]_{\text{原}} & [x]_{\text{原}} &= 1.1111 \\ & & \therefore x &= -0.1111\end{aligned}$$

当真值为~~负~~时, 根据补码求原码的快捷方法

- 补码除符号位外, 末位减 1, 再每位取反
- 自右向左从最先遇到的1左边开始, 每位取反(符号位除外)



*例：已知整数 $[X]_{\text{补}}$ ，求 $[-X]_{\text{补}}$

设 $[X]_{\text{补}} = X_n X_{n-1} \cdots X_1 X_0$ ，根据符号位 X_n 为0或1分开讨论

< I >

$$[X]_{\text{补}} = 0, X_{n-1} X_{n-2} \cdots X_1 X_0$$

< I I >

$$[X]_{\text{补}} = 1, X_{n-1} X_{n-2} \cdots X_1 X_0$$

$$X_{\text{原}} = 0, X_{n-1} X_{n-2} \cdots X_1 X_0$$

$$X_{\text{原}} = 1, (\bar{X}_{n-1} \bar{X}_{n-2} \cdots \bar{X}_1 \bar{X}_0 + 1)$$

$$[-X]_{\text{原}} = 1, X_{n-1} X_{n-2} \cdots X_1 X_0$$

$$[-X]_{\text{原}} = 0, (\bar{X}_{n-1} \bar{X}_{n-2} \cdots \bar{X}_1 \bar{X}_0 + 1)$$

$$[-X]_{\text{补}} = 1, (\bar{X}_{n-1} \bar{X}_{n-2} \cdots \bar{X}_1 \bar{X}_0 + 1)$$

$$[-X]_{\text{补}} = 0, (\bar{X}_{n-1} \bar{X}_{n-2} \cdots \bar{X}_1 \bar{X}_0 + 1)$$

$[X]_{\text{补}}$ 连同符号位在内，每位取反，末位加1，即得 $[-X]_{\text{补}}$

观察发现 $[-X]_{\text{补}} + [X]_{\text{补}} = 2^{n+1} \rightarrow 0 \pmod{2^{n+1}}$ $[-X]_{\text{补}} = -[X]_{\text{补}}$

*例：已知小数 $[y]_{\text{补}}$ ，求 $[-y]_{\text{补}}$

设 $[y]_{\text{补}} = y_0 \cdot y_1 y_2 \dots y_n$ ，根据符号位 y_0 为0或1分开讨论

< I >

$$[y]_{\text{补}} = 0.y_1 y_2 \dots y_n$$

$$-y = -0.y_1 y_2 \dots y_n$$

$$[-y]_{\text{原}} = 1.y_1 y_2 \dots y_n$$

$$[-y]_{\text{补}} = 1 + (0.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n})$$

$$[-y]_{\text{补}} = 1.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n}$$

< II >

$$[y]_{\text{补}} = 1.y_1 y_2 \dots y_n$$

$$[y]_{\text{原}} = 1 + (0.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n})$$

$$y = - (0.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n})$$

$$-y = 0.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n}$$

$$[-y]_{\text{补}} = 0.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n}$$

$[y]_{\text{补}}$ 连同符号位在内，每位取反，末位加 1，即得 $[-y]_{\text{补}}$

观察发现 $[-y]_{\text{补}} + [y]_{\text{补}} = 2 \rightarrow 0 \pmod{2}$ $[-y]_{\text{补}} = -[y]_{\text{补}}$

反码表示法： 二进制整数

负数的原码→补码： 除符号位外每位取反末位加1

$$[x]_{\text{反}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ (2^{n+1}-1) + x & 0 \geq x > -2^n \quad (\text{mod } 2^{n+1}-1) \end{cases}$$

其中： x 为真值， n 为数值位数

$$x = +1101$$

$$[x]_{\text{反}} = 0,1101$$



用 逗号 将符号位
和数值部分隔开

$$x = -1101$$

$$[x]_{\text{反}} = (2^{4+1} - 1) - 1101$$

$$= 11111 - 1101$$

$$= 1,0010$$

$$[x]_{\text{原}} = 1,1101$$

数值位按位取反

反码表示法：二进制小数

小数反码定义：

$$[x]_{\text{反}} = \begin{cases} x & 1 > x \geq 0 \\ (2-2^{-n}) + x & 0 \geq x > -1 \pmod{2-2^{-n}} \end{cases}$$

其中： x 为真值， n 为数值位数

$$x = -0.1010$$

$$[x]_{\text{反}} = (2-2^{-4}) - 0.1010$$

$$= 1.1111 - 0.1010$$

$$= 1.0101$$

用 小数点 将符号位

和数值部分隔开



$$[x]_{\text{原}} = 1.1010$$

数值位按位取反

反码计算的两个例子

- 已知 $[x]_{\text{反}} = 1,1110$ ，求真值 x

解：由定义得 $x = [x]_{\text{反}} - (2^{4+1} - 1)$

$$= 1,1110 - 11111$$
$$= -0001$$

- 求 0 的反码

解：设 $x = +0.0000$ ， $[+0.0000]_{\text{反}} = 0.0000$

$$x = -0.0000, [-0.0000]_{\text{反}} = 1.1111$$

同理，对于整数 $[+0]_{\text{反}} = 0,0000$ ， $[-0]_{\text{反}} = 1,1111$

$$[+0]_{\text{反}} \neq [-0]_{\text{反}}$$

三种机器数（机器码）的小结

- 最高位为符号位，**书写上**用 “,” （整数）或 “.” （小数）将数值部分和符号位隔开
- 对于正数：符号位为0，原码 = 补码 = 反码
- 对于负数：补码 $\longleftrightarrow$ 原码的方法：
 - 除符号位外，**每位取反，末位加 1**
 - **自右向左**从最先遇到的**1**左边开始，**每位取反**(除符号位外)
- 对于负数：补码=反码+1

反码=原码按位取反（符号位除外）

例：机器数的真值

- 设机器数字长为8位(其中1位为符号位);对于整数, 当其分别代表无符号数、原码、补码和反码时, 对应的真值分别为什么?

二进制代码	无符号数 对应的真值	原码对应 的真值	补码对应 的真值	反码对应 的真值
00000000	0	+0	<u>±0</u>	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
⋮	⋮	⋮	⋮	⋮
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
⋮	⋮	⋮	⋮	⋮
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0

移码表示法

- 补码表示很难直接判断其真值大小

如	十进制	二进制	补码	
	$x = +21$	+10101	0,10101	错
	$x = -21$	-10101	1,01011	大
	$x = +31$	+11111	0,11111	错
	$x = -31$	-11111	1,00001	大
以上	$x + 2^5$	$+10101 + 100000 = 110101$		大
		$-10101 + 100000 = 001011$		正确
		$+11111 + 100000 = 111111$		大
		$-11111 + 100000 = 000001$		正确

移码表示法： 二进制整数

- 移码定义

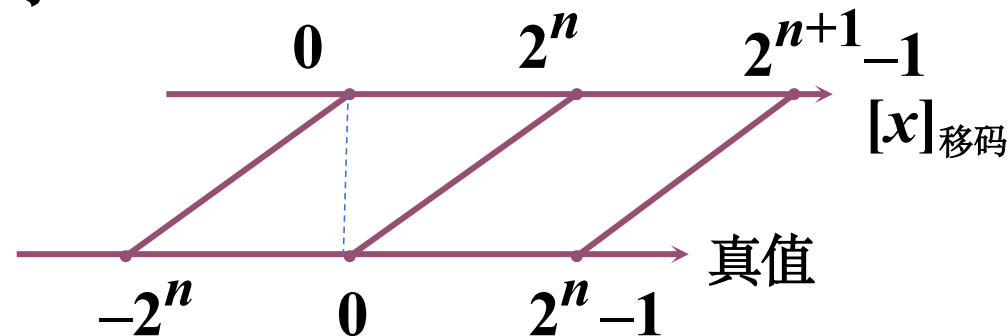
$$[x]_{\text{移}} = 2^n + x \quad (2^n > x \geq -2^n)$$

其中： x 为真值， n 为 整数的位数

小数的移码定义呢？



- 移码在数轴上的表示



- 例：

$$x = 10100$$

$$[x]_{\text{移}} = 2^5 + 10100 = 1,10100$$

$$x = -10100$$

$$[x]_{\text{移}} = 2^5 - 10100 = 0,01100$$

用 逗号 将符号位
和数值位隔开

移码和补码的比较

设 $x = +1100100$

$$[x]_{\text{移}} = 2^7 + 1100100 = \mathbf{1},1100100$$

$$[x]_{\text{补}} = \mathbf{0},1100100$$

设 $x = -1100100$

$$[x]_{\text{移}} = 2^7 + (-1100100) = \mathbf{0},0011100$$

$$[x]_{\text{补}} = 2^{7+1} - 1100100 = \mathbf{1},0011100$$

补码与移码只差一个符号位

真值、补码和移码的对照表

真值 x ($n=5$)	$[x]_{\text{补}}$	$[x]_{\text{移}}$	$[x]_{\text{移}}$ 对应的 十进制整数
- 1 0 0 0 0	1 0 0 0 0	0 0 0 0 0	0
- 1 1 1 1 1	1 0 0 0 1	0 0 0 0 1	1
- 1 1 1 1 0	1 0 0 0 1 0	0 0 0 0 1 0	2
⋮	⋮	⋮	⋮
- 0 0 0 0 1	1 1 1 1 1 1	0 1 1 1 1 1	31
± 0 0 0 0 0	0 0 0 0 0 0	1 0 0 0 0 0	32
+ 0 0 0 0 1	0 0 0 0 0 1	1 0 0 0 0 1	33
+ 0 0 0 1 0	0 0 0 0 1 0	1 0 0 0 1 0	34
⋮	⋮	⋮	⋮
+ 1 1 1 1 0	0 1 1 1 1 0	1 1 1 1 1 0	62
+ 1 1 1 1 1	0 1 1 1 1 1	1 1 1 1 1 1	63

移码的特点

续前表, $n=5$

$$[+0]_{\text{移}} = 2^5 + 0 = 1,00000$$

$$[-0]_{\text{移}} = 2^5 - 0 = 1,00000$$

$$[+0]_{\text{移}} = [-0]_{\text{移}}$$

最小真值 $-2^5 = -100000$ 对应的移码为 $2^5 - 100000 = 000000$

最小真值的移码为全 0



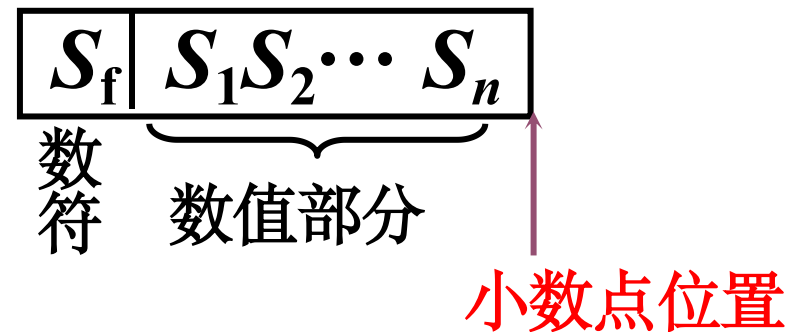
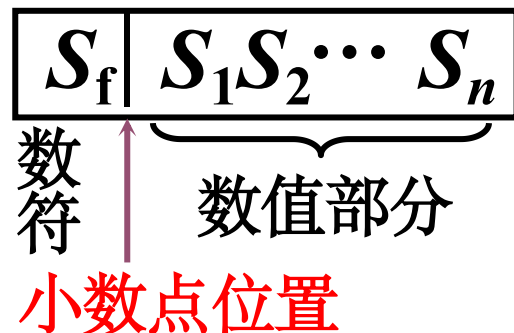
可用**移码思想**表示浮点数的阶码, 便于判断浮点数阶码大小

计算机中数的表示

- 无符号数和有符号数
- **定点表示和浮点表示**
- IEEE754标准

定点表示

- 小数点按约定方式标出



定点机

小数定点机

整数定点机

原码

$$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$$

$$-(2^n - 1) \sim +(2^n - 1)$$

补码

$$-1 \sim +(1 - 2^{-n})$$

$$-2^n \sim +(2^n - 1)$$

反码

$$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$$

$$-(2^n - 1) \sim +(2^n - 1)$$

浮点表示

$N = S \times r^j$ 浮点数的一般形式

S 尾数 j 阶码 r 基数 (基值)

计算机中 r 取 2、4、8、16 等

当 $r = 2$

$$N = 11.0101$$

二进制表示

$$\checkmark = 0.110101 \times 2^{10}$$

规格化数

$$= 1.10101 \times 2^1$$

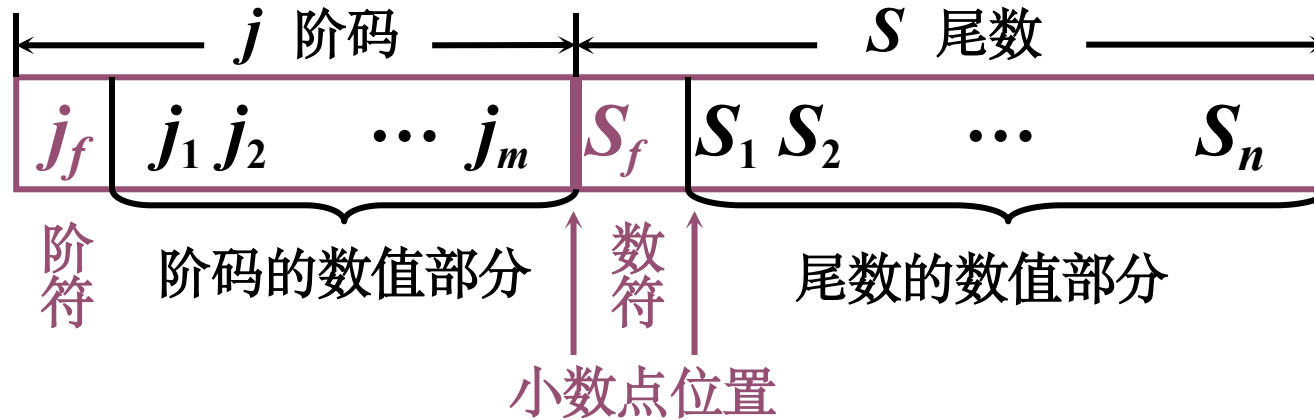
$$= 1101.01 \times 2^{-10}$$

$$\checkmark = 0.00110101 \times 2^{100}$$

计算机中 S 小数、可正可负

j 整数、可正可负

浮点数的表示形式

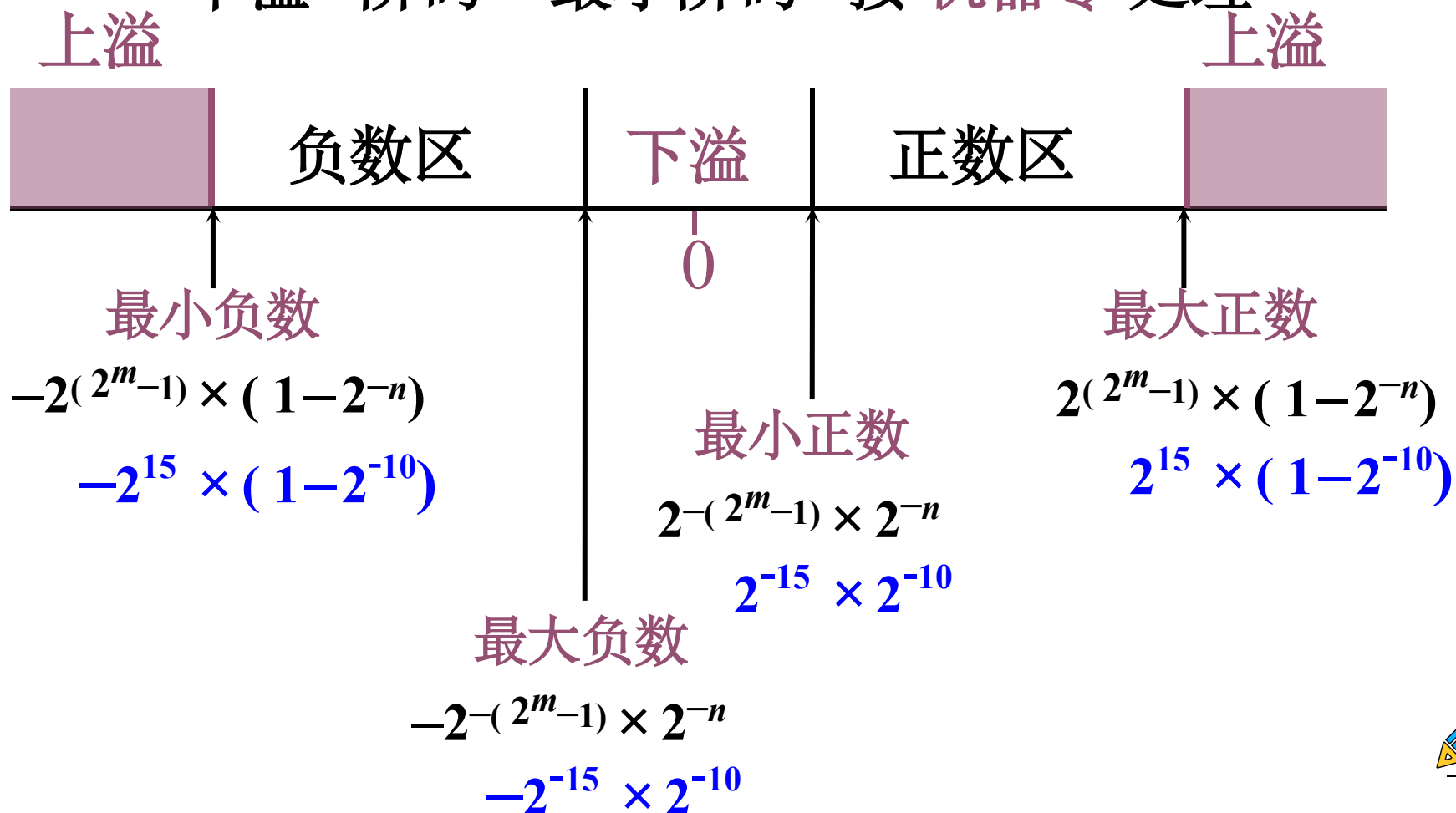


- j_i, S_i 取0或1 (现代计算机中采用二进制数)
- S_f 代表浮点数的符号
- n 尾数数值位数, 反映浮点数的精度
- m 阶码数值位数, 反映浮点数的表示范围
- j_f 和 m 共同表示小数点的实际位置

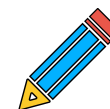
浮点数的表示范围 (真值)

上溢 阶码 > 最大阶码

下溢 阶码 < 最小阶码 按机器零处理



设 $m = 4$
 $n = 10$



练习

- 设机器数字长为 24 位，欲表示 ± 3 万的十进制数，试问在保证数的**最大精度**的前提下，除阶符、数符各取1位外，阶码数值、尾数数值各取几位？

解： $\because 2^{14} = 16384 \quad 2^{15} = 32768$

$\therefore$ 至少**15** 位二进制数才能表示 ± 3 万之间的十进制数

$$\begin{array}{c} 2^{15} \times 0.\underbrace{\times \times \times \dots \times \times \times}_{15\text{位}} \\ \uparrow \\ m \geq 4 \end{array}$$

满足 最大精度： $m = 4, n = 18$

浮点数规格化

- 基数不同，浮点数的规格化形式不同

$r = 2$ 尾数最高位为 1 (不为0)

尾数和阶码位数保持不变的情况下,

$r = 4$ 尾数最高 2 位不全为 0

基数 r 越大, 可表示的浮点数:

$r = 8$ 尾数最高 3 位不全为 0

- 范围越大
- 精度降低

- ## • 浮点数的规格化

$r = 2$ 左规 尾数左移 1 位, 阶码减 1

右规 尾数右移 1 位, 阶码加 1

$r = 4$ 左规 尾数左移 2 位, 阶码减 1

右规 尾数右移 2 位, 阶码加 1

$r = 8$ 左规 尾数左移 3 位, 阶码减 1

右规 尾数右移 3 位, 阶码加 1

二进制浮点数规格化

(1) 二进制($r=2$)规格化数的定义

$$1/2 \leq |S| < 1$$

(2) 规格化数的判断

$S > 0$	规格化形式	$S < 0$	规格化形式
真值	$0.1 \times \times \dots \times$	真值	$-0.1 \times \times \dots \times$
原码	0.1 $\times \times \dots \times$	原码	1.1 $\times \times \dots \times$
补码	0.1 $\times \times \dots \times$	补码	1.0 $\times \times \dots \times$
反码	$0.1 \times \times \dots \times$	反码	$1.0 \times \times \dots \times$

机器判断方法:

原码	不论正数、负数, 第一数位为1
补码	符号位和第 1 数位不同

二进制数规格化的特例

$$S = -\frac{1}{2} = -0.100 \cdots 0$$

$$[S]_{\text{原}} = 1.100 \cdots 0$$

$$[S]_{\text{补}} = \boxed{1.1}00 \cdots 0$$

$\therefore [-\frac{1}{2}]_{\text{补}}$ 按照定义是规格化数，而机器判断不是规格化数

$$S = -1.0$$

$$[S]_{\text{补}} = \boxed{1.0}00 \cdots 0$$

$\therefore [-1]_{\text{补}}$ 按照定义不是规格化数，而机器判断是规格化数

浮点数规格化实例

- 假设阶码与尾数都是**原码**表示($r=2$),阶码数值位数 $m=4$,尾数数值位数 $n=10$, 求**规格化**浮点数的表示范围。

$$\text{最大正数} \quad 2^{+1111} \times \underbrace{0.1111111111}_{10 \text{ 个 } 1} = 2^{15} \times (1 - 2^{-10})$$

$$\text{最小正数} \quad 2^{-1111} \times \underbrace{0.1000000000}_{9 \text{ 个 } 0} = 2^{-15} \times 2^{-1} = 2^{-16}$$

$$\text{最大负数} \quad 2^{-1111} \times (-\underbrace{0.1000000000}_{9 \text{ 个 } 0}) = -2^{-15} \times 2^{-1} = -2^{-16}$$

$$\text{最小负数} \quad 2^{+1111} \times (-\underbrace{0.1111111111}_{10 \text{ 个 } 1}) = -2^{15} \times (1 - 2^{-10})$$

小数转浮点数实例

- 将 $+\frac{19}{128}$ 写成二进制定点数、浮点数；并分别写出其在16位定点机和浮点机中的机器数形式（原码、补码、反码、浮点数）。假设：尾数数值部分10位，数符1位，阶码5位(含1位阶符)。

解： 设 $x = +\frac{19}{128}$

二进制真值 $x = +0.0010011$

浮点数表示 $x = 0.0010011$

浮点规格化形式 $x = 0.10011 \times 2^{-10}$

定点机(16位) $[x]_{\text{原}} = [x]_{\text{补}} = [x]_{\text{反}} = 0.0010011\textcolor{red}{00000000}$

浮点机(16位) $[x]_{\text{原}} = 1, 0010 \quad 0.1001100000$

$[x]_{\text{补}} = 1, 1110 \quad 0.1001100000$

$[x]_{\text{反}} = 1, 1101 \quad 0.1001100000$

整数转浮点数实例

- 将-58表示成二进制定点数和规格化浮点数，并写出它在定点机和浮点机中的机器数表示。假设：定点机字长11位；浮点机字长16位，尾数为补码表示，数值位10位，数符1位，阶码5位(移码表示，含1位阶符)。

解： 设 $x = -58$

二进制真值

$$x = -111010$$

定点表示

$$x = -0000111010$$

浮点规格化形式 $x = -(0.1110100000) \times 2^{110}$

11位定点机中

$$[x]_{\text{原}} = 1, 0000111010$$

$$[x]_{\text{补}} = 1, 1111000110$$

$$[x]_{\text{反}} = 1, 1111000101$$

16位浮点机中

$$[x]_{\text{原}} = 0, 0110; 1. 1110100000$$

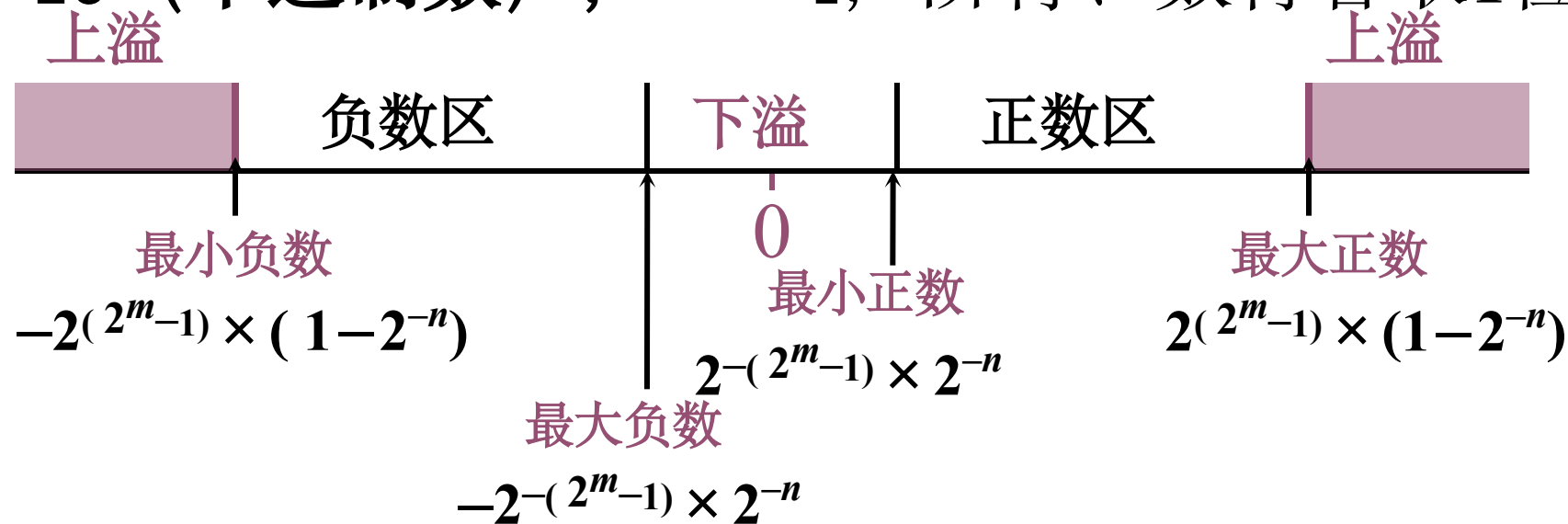
$$[x]_{\text{补}} = 0, 0110; 1. 0001100000$$

$$[x]_{\text{反}} = 0, 0110; 1. 0001011111$$

$$[x]_{\text{阶移、尾补}} = 1, 0110; 1. 0001100000$$

例：写出真值所对应的浮点数补码表示

- 设 $n=10$ (十进制数)， $m=4$ ，阶符、数符各取1位。

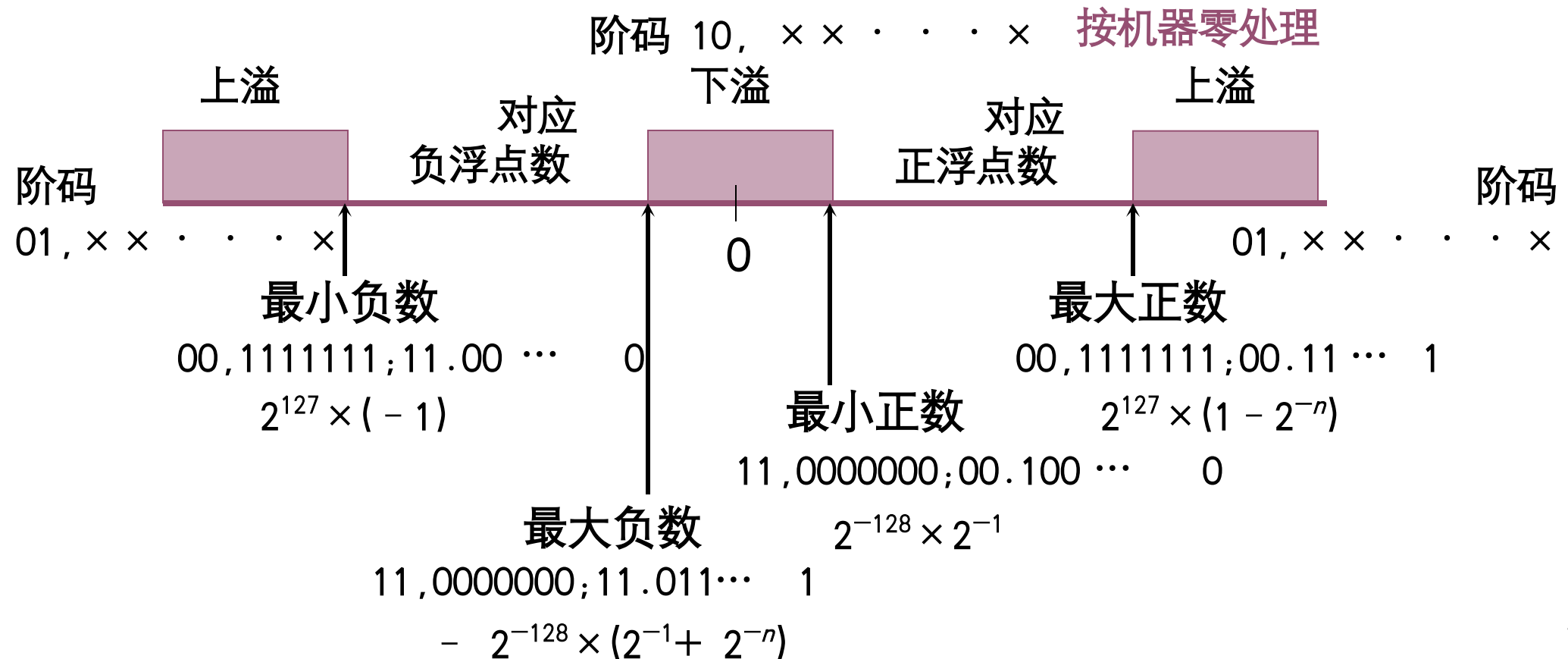


解：

	真值	补码
最大正数	$2^{15} \times (1-2^{-10})$	0,1111; 0.1111111111
最小正数	$2^{-15} \times 2^{-10}$	1,0001; 0.0000000001
最大负数	$-2^{-15} \times 2^{-10}$	1,0001; 1.1111111111
最小负数	$-2^{15} \times (1-2^{-10})$	0,1111; 1.0000000001

补码表示的浮点数范围

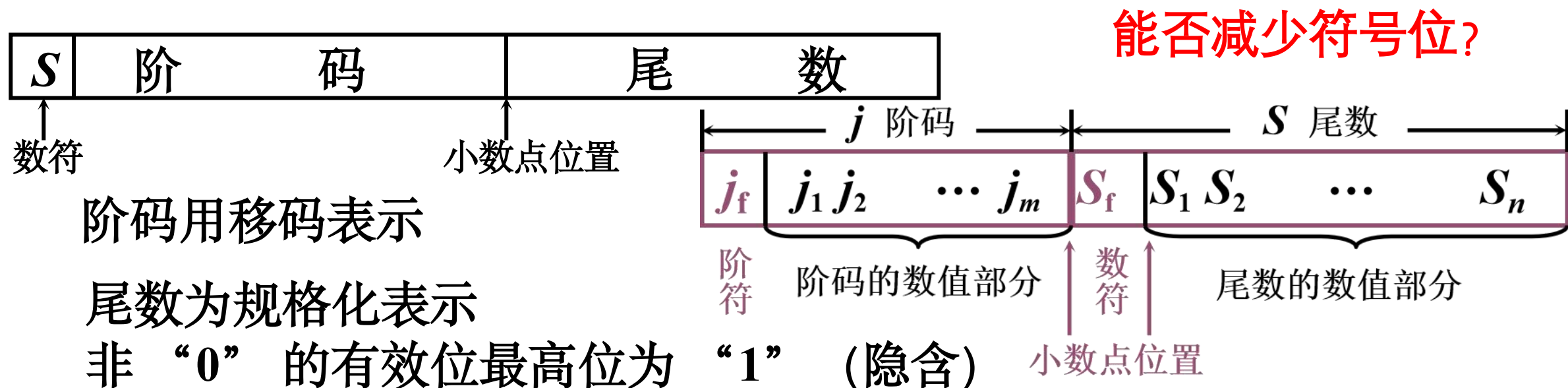
- 设机器数为补码，尾数为规格化形式，并假设阶符取2位，阶码数值位为7位，数符取2位，尾数数值位取 n 位，则该补码在数轴上的表示为：



第二章 计算机中数的表示

- 无符号数和有符号数
- 定点表示和浮点表示
- IEEE754标准

IEEE 754 标准



	符号位 S	阶码	尾数	总位数
短实数	1	8	23	32 (单精度)
长实数	1	11	52	64 (双精度)
临时实数	1	15	64	80

IEEE 754浮点数标准

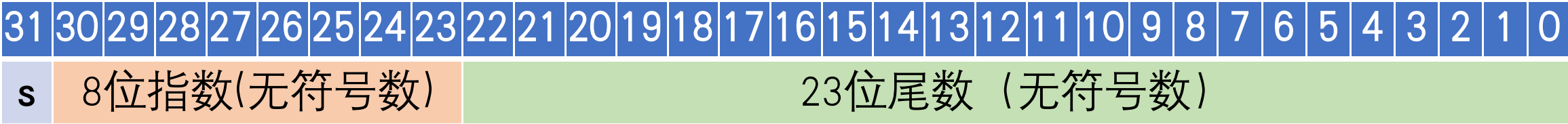
- 单精度 (32-bit)



- 双精度 (64-bit)



IEEE 754浮点数：单精度为例



指数	尾数	表示对象	换算方法
0	0	正负0	规定 (符号位不同, 存在+0.0和-0.0)
0	非0	正负非规格化数	正负非规格化数 = $(-1)^s * (\text{尾数}_2) * 2^{(0 - 126)}$ (S代表符号位, 1为负数, 0为正数)
[1: 254]	任意	正负(规格化)浮点数	正负浮点数 = $(-1)^s * (1 + \text{尾数}_2) * 2^{(\text{指数} - 127)}$
255	0	正负无穷 (inf)	规定
255	非零	NaN	规定

IEEE 754浮点数： 正负浮点数

- 正负浮点数 = $(-1)^s \times (\underline{1 + \text{尾数}_2}) \times 2^{(\text{指数} - 127)}$
- 尾数前加一？
 - 这个1称为**前导数**。为了打包更多的位到数中，就在二进制表示中省略了前导数，默认小数点前有1（硬件自动附加1）。
 - **有效位数**：尾数位数+1，对单精度来说，有效位数24位（隐含的1和23位尾数）；对双精度，有效位数53位（1+52）。
 - 由于0（和非规格化数）没有前导数，所以被赋予特殊的指数0，硬件不会给尾数附加1

IEEE754浮点数： 正负浮点数

- 正负浮点数 = $(-1)^s \times (1 + \text{尾数}_2) \times 2^{(\text{指数} - 127)}$
- 指数为何减 127?
 - 使用**移码**的思想。二进制表示中的指数部分是无符号数，可以直接进行大小比较。如果两个数的符号相同，那么具有更大二进制指数的数就更大。
 - 对于真值，实际“指数”范围： $[1-127: 254-127] = [-126: 127]$
 - 如果减128，实际“指数”范围： $[1-128: 254-128] = [-127: 126]$
- IEEE754浮点数在坐标轴上是否**均匀**分布？
 - 阶码（指数）相同的浮点数区间内均匀分布
 - 不同指数的浮点数区间，间隔不同（分布不均匀）



IEEE 754浮点数： 正负非规格化数

- 正负非规格化数 = $(-1)^S \times (\text{尾数}_2) \times 2^{(0 - 126)}$
- 什么是非规格化数？
 - 规格化数：尾数部分最左边（小数点右边第一位）不为 0 的数
 - 非规格化数：尾数部分最左边为 0 的数
- 非规格化数的绝对值比（规格化）浮点数绝对值更小
 - 对于正负（规格化）浮点数来说，若二进制指数部分为1，则真值指数部分为 -126，和非规格化数相同。
 - 但（规格化）浮点数尾数有前导1，导致浮点数绝对值更大。

IEEE 754浮点数： 正负非规格化数

- 正负浮点数 = $(-1)^S \times (1 + \text{尾数}_2) \times 2^{(\text{指数} - 127)}$ (指数: 1——254)
- 正负非规格化数 = $(-1)^S \times (\text{尾数}_2) \times 2^{(0 - 126)}$

最小正浮点数: $N_1 = (1 + 0_2) * 2^{(1 - 127)} = \mathbf{2^{-126}}$

第二小正浮点数: $N_2 = (1 + 0.0...01_2) * 2^{(1-127)} = \mathbf{2^{-126} + \underline{2^{-149}}}$



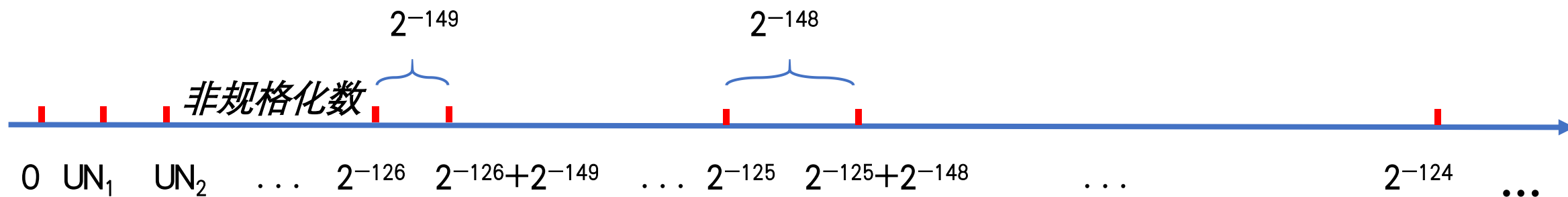
最大非规格化数: $UN_x = 0.1...11_2 * 2^{(0 - 126)} = (1 - 2^{-23}) * 2^{-126} = \mathbf{2^{-126} - \underline{2^{-149}}}$

.....

最小正非规格化数: $UN_1 = 0.0...01_2 * 2^{(0 - 126)} = 2^{-23} * 2^{-126} = \mathbf{\underline{2^{-149}}}$

IEEE754浮点数小结

- IEEE 754定义非规格化数的指数为-126，主要是因为最小的正规格化浮点数为 2^{-126} ，在 $(0, 2^{-126})$ 内，相邻两浮点数间隔为 $2^{-126} * 2^{-23}$ 。
- 阶码（指数）相同的浮点数区间内均匀分布，不同指数的浮点数区间，间隔不同（分布不均匀）。
- IEEE 754单精度浮点数不同区间的分布， n 为指数，23为尾数的位数。



$2^{n-127}, 2^{n-127+1}$ 区间相邻两单精度浮点数的间隔：

$$2^{n-127} * 2^{-23} = 2^{n-150} \quad (1 \leq n \leq 254)$$

浮点计算不准确

$a = 0.1 + 0.2$

$b = 0.3$

`print(a == b)` # 输出: True?False?

`print(a)` # 输出: 0.30000000000000004

- 二进制表示的限制:
 - 不是所有的十进制小数都能精确地表示为二进制小数。例如，十进制小数0.1, 3.14等。
- IEEE 754标准局限性（计算机）：
 - 无法完全精确地表示所有实数，尤其是在处理非常大或非常小的数值时。

IEEE 754浮点数：真值转二进制

- 例:将十进制 -0.75 转为单精度IEEE754浮点数。
- 解:十进制小数转二进制小数: $-0.75_{10} = -0.11_2$
规格化: $-0.11 = -1.1 \times 2^{-1}$, 能够规格化, 说明是正负浮点数表示
$$\underline{-1.1} \times 2^{-1} = (-1)^S \times (\underline{1 + \text{尾数}_2}) \times 2^{(\text{指数} - 127)}$$
$$= (-1)^{\textcolor{red}{1}} \times (1 + 0.\textcolor{blue}{1}_2) \times 2^{(\textcolor{red}{126} - 127)}$$

符号位: 1; 指数部分: 126; 尾数部分: 0.1_2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

IEEE 754浮点数： 二进制转真值

例： 将二进制IEEE754浮点数表示转换为十进制浮点数。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

解

符号位为1， 指数字段为129， 尾数字段为 $2^{-2} = 0.25$ —>正负浮点数

$$(-1)^S \times (1 + \text{尾数}_2) \times 2^{(\text{指数} - 127)} = (-1)^1 \times (1 + 0.25) \times 2^{(129 - 127)}$$

$$= -1 \times 1.25 \times 2^2$$

$$= -5.0$$

IEEE 754浮点数转换工具

This page allows you to convert between the decimal representation of numbers (like "1.02") and the binary format used by all modern CPUs (IEEE 754 floating point).

[illegible]

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>